



**AIRSHIELD**

PRIVATE AI CONTROL PLANE

ARCHITECTURE • SECURITY • ASSURANCE

# Technical Authority Dossier

Implemented architecture, security controls, cryptography, protocols, evidence and production gates

**IMPLEMENTED FACTS. EXPLICIT  
BOUNDARIES. FAIL-CLOSED GATES.**

CAPTURE

PROTECT

AUTHORIZE

PROVE

# Contents

Implemented architecture, security controls, cryptography, protocols, evidence and production gates

---

|                                                   |    |
|---------------------------------------------------|----|
| 1. Document authority and scope                   | 3  |
| 2. System objective                               | 3  |
| 3. Implemented capability inventory               | 4  |
| 4. Logical architecture                           | 4  |
| 5. Component and technology inventory             | 6  |
| 6. UI and user-experience implementation          | 6  |
| 7. Text-protection data flow                      | 7  |
| 8. Voice capture and finalization                 | 7  |
| 9. Participant authenticity and speaker mapping   | 8  |
| 10. Detection and policy model                    | 8  |
| 11. ContextFence cumulative-risk model            | 9  |
| 12. Destination Switch model                      | 9  |
| 13. EgressSeal protocol                           | 10 |
| 14. SafeAction broker demonstration               | 11 |
| 15. Token-vault cryptography                      | 12 |
| 16. Evidence, deletion and reidentification       | 13 |
| 17. Authentication, authorization and tenancy     | 14 |
| 18. API and integration surface                   | 14 |
| 19. Deployment architecture                       | 15 |
| 20. Security-control summary                      | 15 |
| 21. Threat model and residual risk                | 16 |
| 22. Compliance and regulatory posture             | 17 |
| 23. Validation evidence for this baseline         | 17 |
| 24. Production limitations and required decisions | 18 |
| 25. Recommended authority gates                   | 18 |
| 26. Recommended next implementation sequence      | 19 |
| 27. Authority conclusion                          | 19 |

# 1. Document authority and scope

---

This dossier is the technical review starting point for architecture, security, privacy, platform, model-risk, operations and engineering authorities. It covers:

- responsive Next.js product and server routes;
- Python control plane and authorization model;
- self-hosted voice edge and authenticated gateway;
- detection and sector policy;
- token vault and cryptographic providers;
- evidence, retention, deletion and controlled reidentification;
- EgressSeal™, Destination Switch, ContextFence™ and SafeAction™;
- CareShield Assistant;
- contracts, language integrations and deployment references;
- validation evidence, known limitations and production gates.

The repository is a production-shaped hackathon implementation, not production approval. It is not a compliance certificate, legal opinion, medical device, emergency service or real booking/financial/claims platform. The ™ marks denote product-concept branding, not registered trademark status.

## 2. System objective

---

AirShield is designed to ensure that an AI or downstream application receives only policy-permitted protected meaning. The governing transaction is:

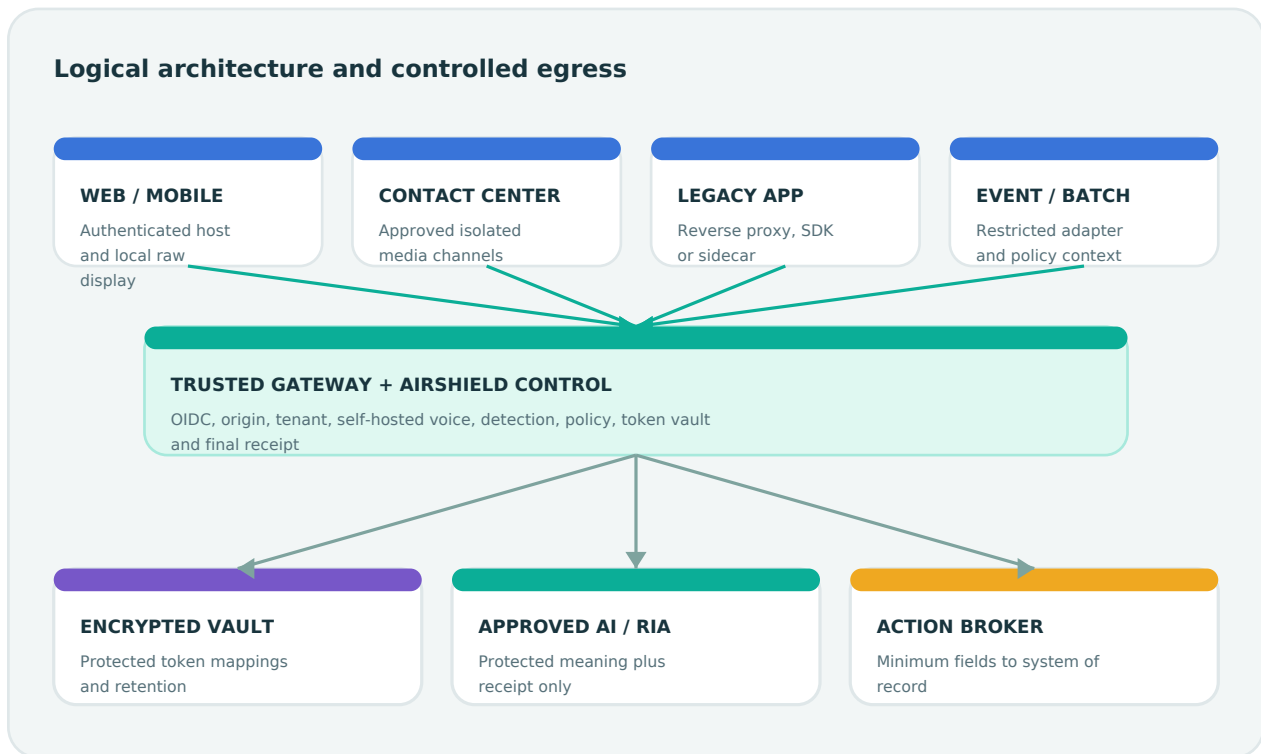
- 1 authenticate the workload and participant outside the model;
- 2 establish tenant, session, policy and exact destination;
- 3 capture or receive English voice/text within the customer boundary;
- 4 detect PII, PHI, PCI data and enterprise secrets;
- 5 mask, tokenize, block or review by policy;
- 6 evaluate cumulative context/linkage risk;
- 7 create verifiable evidence;
- 8 bind release authorization to protected content and destination;
- 9 permit only one seal-authorized token-aware action;
- 10 prevent raw identity from returning to the AI.

The security claim is pre-egress control and proof—not perfect detection, perfect anonymity or perfect endpoint security.

### 3. Implemented capability inventory

| Capability                          | Implementation status                                                          | Production status                                                         |
|-------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Responsive product UI               | Implemented for desktop, tablet and mobile                                     | Requires accessibility/customer UX validation                             |
| Live raw/protected transcript proof | Implemented for sample and self-hosted voice paths                             | Requires representative latency/quality qualification                     |
| Text protection proxy               | Implemented with development detector and control-plane proxy path             | Production requires evaluated detector and authenticated control plane    |
| Python control plane                | Implemented with policy, auth, vault, evidence and reidentification operations | Requires target infrastructure validation and independent review          |
| Voice edge                          | Implemented with self-hosted English ASR adapter and optional diarization      | Requires approved models, hardware, scale and codec evaluation            |
| Authenticated voice gateway         | Implemented design/service with OIDC, origin, tenant and protocol controls     | Requires customer identity, certificates and external exposure validation |
| Token vault                         | Implemented AES-256-GCM envelope-encryption pattern                            | Requires managed key custody, versioning and full rotation/DR tests       |
| Evidence                            | Implemented receipt/hash-chain/provider-signing pattern                        | Requires independent immutable anchoring and operating procedure          |
| EgressSeal                          | Implemented process-local receipt registry and Ed25519 protocol                | Development only; production KMS/HSM and receipt authority required       |
| Destination Switch                  | Implemented route rebinding and risk recalculation                             | Requires authoritative destination registry and network enforcement       |
| ContextFence                        | Implemented explainable deterministic risk signal                              | Requires representative evaluation and governed thresholds                |
| SafeAction                          | Implemented seal-gated synthetic token-only broker                             | Requires separately deployed broker and real scoped connectors            |
| CareShield                          | Implemented protected virtual-intake/synthetic booking UX                      | No real diagnosis, EHR or appointment integration                         |
| Azure/Kubernetes assets             | Reference assets implemented                                                   | Live subscription/cluster, policy, quota and DR validation required       |
| Multi-language SDK examples         | Java, .NET, Python, Node.js and Go examples                                    | Requires supported SDK release lifecycle                                  |

### 4. Logical architecture



## Channel layer

- Next.js web/mobile-responsive UI
- authenticated host applications
- approved telephony/contact-center media adapters
- SDK/proxy/sidecar integrations for legacy applications
- restricted batch/event adapters

## Boundary and processing layer

- authenticated WebSocket gateway
- self-hosted English voice edge
- Next.js server-only routes
- Python FastAPI control plane
- policy and detector stack
- identity-binding service

## Trust and evidence layer

- PostgreSQL tenant records
- encrypted token mappings
- Azure Key Vault or OpenBao cryptographic providers
- signed evidence chain
- retention/deletion and reidentification controls

## Destination and action layer

- approved AI/RIA destinations
- EgressSeal release authorization
- SafeAction broker
- scoped system-of-record connectors

- response/output protection target

## 5. Component and technology inventory

| Component          | Technology                                          | Responsibility                                                            |
|--------------------|-----------------------------------------------------|---------------------------------------------------------------------------|
| Product web        | Next.js 16.3.1, React 19.2.8, TypeScript 5.8        | Responsive UI, server proxy, local browser capture                        |
| UI presentation    | Repository CSS and Lucide                           | Offline-compatible responsive experience                                  |
| Next server routes | Node runtime                                        | Session, protect, health, summarize and development EgressSeal APIs       |
| Control plane      | Python 3.12 target, FastAPI, Pydantic               | Authorization, policy, protection, vault and evidence                     |
| Database           | Async SQLAlchemy, Alembic, PostgreSQL               | Tenant-scoped state and migrations                                        |
| Detection          | Deterministic recognizers, Presidio, spaCy          | Entity detection and context-aware policy input                           |
| Voice edge         | faster-whisper, optional pyannote.audio             | Self-hosted ASR and speaker continuity                                    |
| Voice gateway      | FastAPI/WebSocket                                   | OIDC, origin/tenant/protocol/limit enforcement and private edge mediation |
| Vault crypto       | <a href="#">cryptography</a> AESGCM                 | Per-record authenticated encryption                                       |
| Key providers      | Azure Key Vault, OpenBao Transit, local development | DEK wrapping/unwrapping and signing abstraction                           |
| Evidence           | Hash chain and external signing provider            | Metadata-only decision evidence                                           |
| Contracts          | OpenAPI and AsyncAPI                                | Synchronous and real-time integration contract                            |
| Deployment         | Docker, Compose, Kubernetes/Kustomize, Azure Bicep  | Local and production-reference packaging                                  |
| Optional AI        | Ollama adapter                                      | Self-hosted protected-content demonstration destination                   |

## 6. UI and user-experience implementation

### Overview

Shows synthetic privacy posture, protected-session/entity metrics, local-protection path, traffic and recent protected events. Provides direct navigation to Live Shield and EgressSeal.

### Live Shield

Shows English raw/local and protected/outbound transcripts side by side, sector policy selection, speaker-to-person mapping, host-assurance explanation, self-hosted model path, downstream route and protection API demonstration. Interim output remains visually provisional.

### EgressSeal control room

Combines the four-priority transaction:

- select sector and synthetic raw input;
- switch among private, managed, research and blocked public destinations;
- protect through [/api/protect](#);
- display protected result and upstream receipt;
- display ContextFence score/factors;
- issue and verify EgressSeal;
- unlock and run SafeAction;
- show fail-closed outcome after destination or action mismatch.

## CareShield Assistant

An initially open, collapsible widget available across views. It supports protected typed input, self-hosted voice capture, emergency language, local raw bubble, protected RIA preview, synthetic doctor slot and synthetic reservation. Review/block results do not open the booking step. Voice requires a signed final allow before opening the protected downstream flow.

## Governance and operations views

Policy Studio, Token Vault, Audit Trail, Connections, Performance Lab and Settings demonstrate policy management, controlled reidentification, evidence, integrations, evaluation and private deployment concepts.

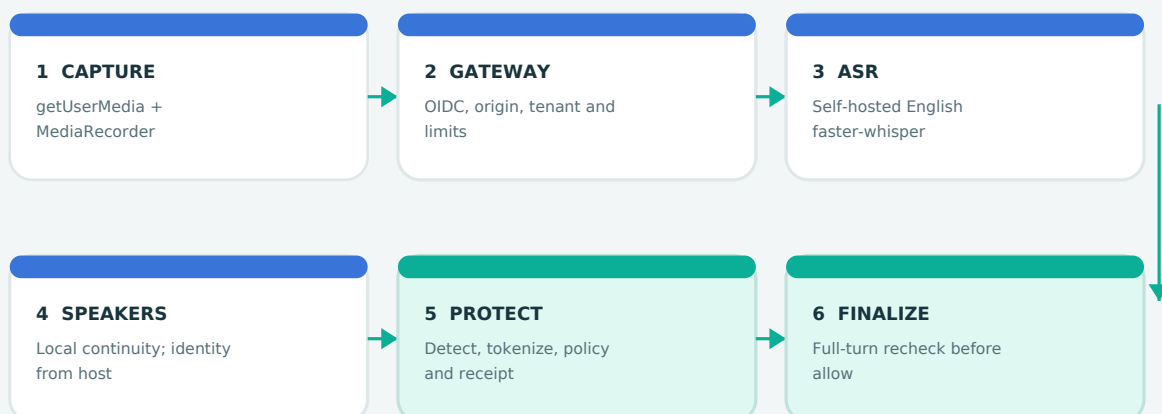
# 7. Text-protection data flow

- 1 The host establishes an authenticated user/workload context.
- 2 The browser sends text to a same-origin Next.js route.
- 3 The server ignores browser-supplied bearer identity for workload calls.
- 4 The server creates/reuses a tenant policy session.
- 5 In configured mode, it calls the Python control plane using workload identity or an allowed development token.
- 6 The control plane validates tenant, permission, session, policy and destination.
- 7 Detectors produce non-overlapping sensitive spans.
- 8 Policy selects mask, tokenize, block or review behavior.
- 9 Reversible tokens receive encrypted mappings.
- 10 The protected text and metadata-only receipt are returned.
- 11 In explicit UI-only development mode, the Next server uses a labelled deterministic detector and `demo_unsigned` receipt.
- 12 Production mode without a control plane returns denial/service unavailable rather than silently falling back.

The Next EgressSeal route is disabled in production because its embedded signer is not an external trust authority.

# 8. Voice capture and finalization

## Voice path: interim is provisional; final allow governs egress



## Browser

- `getUserMedia` requests one audio track.
- `MediaRecorder` selects a supported WebM/Opus, WebM, MP4 or Ogg representation.
- A WebSocket sends `session.start`, bounded binary chunks and `session.end`.
- No browser/vendor speech-recognition fallback is used.
- Stop releases media tracks and waits for final protection.

## Gateway

The production gateway validates host OIDC cookie/bearer, issuer, audience, signature, expiry, tenant and scopes/roles. It validates exact Origin, protocol sequence, message/frame size, rate, connections and session duration. It discards client trust headers, injects a private rotatable edge secret and validates private edge TLS/CA.

## Edge

The voice edge runs self-hosted English ASR and optional diarization. Transcript-pair events are local/provisional and `safe_for_egress=false`. At `session.end`, remaining short audio is forced through transcription, the complete transcript is rechecked and a final decision/receipt is emitted.

A final allow is intended to require a non-demo signature and explicit `safe_for_egress=true`. Development local voice can remain review/unsigned and must not be described as production egress authorization.

# 9. Participant authenticity and speaker mapping

---

The system separates identity, continuity and liveness:

- 1 Host application authenticates the participant through SSO, OTP, portal, CRM/EHR check-in, IVR or another approved mechanism.
- 2 Host establishes isolated-channel provenance when available.
- 3 Host issuer creates a signed session/track binding claim.
- 4 Gateway verifies issuer, audience, tenant, expiry, session and scope.
- 5 Control plane stores a keyed subject digest and returns an unrelated speaker token.
- 6 Diarization maintains track continuity but does not prove identity.
- 7 Missing/ambiguous provenance remains unknown.

Anti-spoof/liveness is a separate control. Depending on risk, the host may require device binding, challenge-response, telephony attestation, supervised enrollment or specialist services. AirShield consumes assurance; it does not derive trusted identity from transcript names or voice similarity.

# 10. Detection and policy model

---

Implemented policy packs cover:

- Healthcare US/EU
- Finance US/EU
- Insurance US/EU
- Contact Center EU/US
- SaaS/Internal Copilot EU/US

Policy includes entity handling, destination and release behavior. Deterministic recognizers handle structured patterns; Presidio/spaCy provide contextual detection in the production-shaped control plane. Production can fail startup/routing when required models are unavailable.



Detector security limitations:

- no detector provides perfect recall;
- ASR errors can hide entities before detection;
- safe-looking facts can combine into identity;
- over-detection can remove required utility;
- domain, accent, noise, codec and adversarial data require representative evaluation;
- output/TTS and tool parameters need equivalent downstream policy.

## 11. ContextFence cumulative-risk model

ContextFence is implemented in `lib/context-fence.ts` and used by the EgressSeal route/UI. It is deterministic and explainable.

### Current factors

| Factor                     | Current signal                                                 |
|----------------------------|----------------------------------------------------------------|
| Destination exposure       | Private, managed, restricted or external baseline              |
| Stable-token linkage       | Count of protected references that can link turns/records      |
| Entity combination         | Diversity of entity types in one context                       |
| Sensitive semantic context | Health, account, claim, transaction, incident and role meaning |
| Quasi-identifier detail    | Timing, scheduling, age, dose and measurements                 |
| Context accumulation       | Longer protected content increases mosaic risk                 |
| Relationship graph         | Family, witness, clinician, supervisor and other role links    |

The result is bounded 0–100 with low, guarded, high and critical bands and an allow/review/block disposition. Destination profiles have different base risk and maximum threshold.

### Authority limitation

ContextFence does not prove k-anonymity, differential privacy or non-identifiability. Current weights are product-demonstration policy values. Production requires representative linkage attacks, calibration, false-positive/negative review, sector governance, threshold versioning and monitoring.

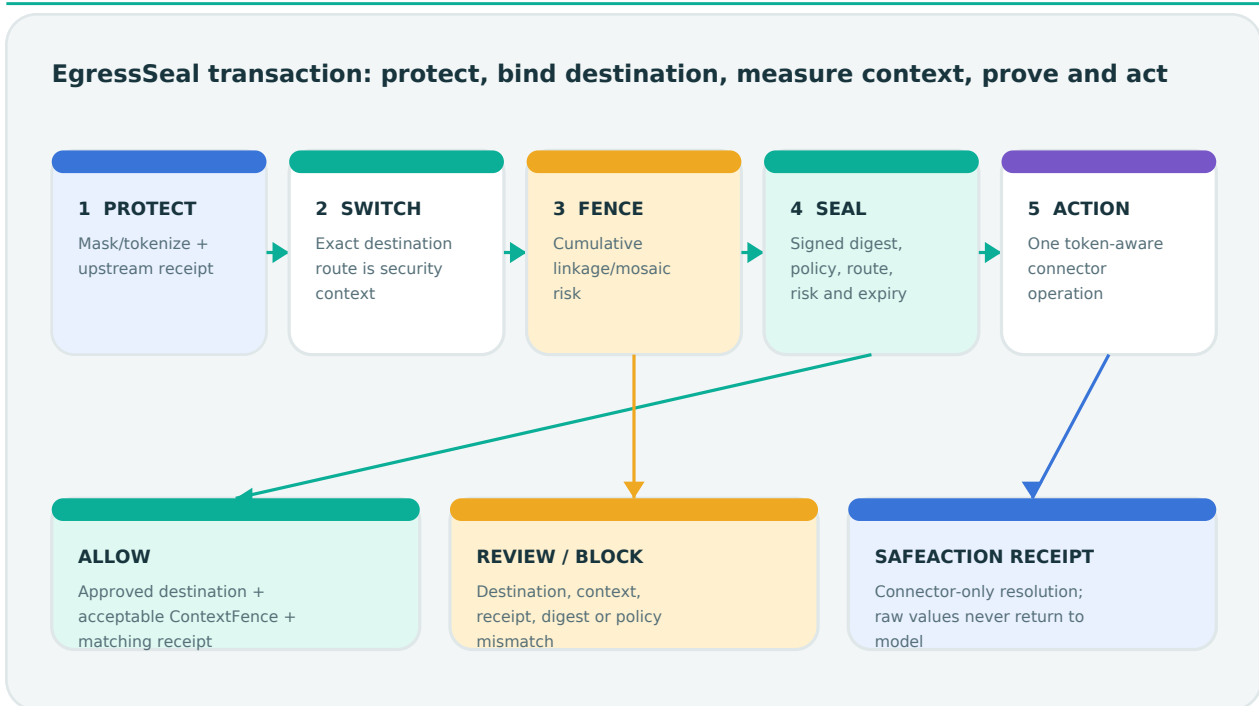
## 12. Destination Switch model

Implemented destination profiles are:

| Destination             | Trust/status             | Demonstration behavior                            |
|-------------------------|--------------------------|---------------------------------------------------|
| Organization private AI | Private / approved       | Lowest base risk; sector-approved route           |
| Regional managed RIA    | Managed / approved       | Higher exposure and distinct bound route          |
| Research sandbox        | Restricted / conditional | Lower ContextFence threshold and review potential |
| Public general AI       | External / blocked       | EgressSeal withheld regardless of field masking   |

A destination change clears the displayed seal/action state and requires a new `/api/protect` receipt for the newly selected route. The prior seal remains bound to its original destination until expiry; it cannot authorize the newly selected destination. Production additionally requires authoritative destination identity, registry, network egress policy and revocation.

## 13. EgressSeal protocol



### Development issuance prerequisites

The Next protection route registers a process-local development receipt containing:

- receipt identifier;
- SHA-256 of protected text;
- decision;
- policy;
- destination route;
- 15-minute process-local expiry.

EgressSeal issuance checks that the browser-provided candidate matches this server-side record. This prevents a client from inventing an allow decision or reusing a receipt for changed content, policy or destination within the development process.

### Signed payload

The development seal binds:

- protocol version `egressseal/v1`;
- random seal ID;
- protected-content SHA-256;
- upstream receipt ID;
- policy;
- destination profile ID, label and route;
- ContextFence score/band;
- one policy-derived allowed action;
- issue and ten-minute expiry timestamps;
- random nonce;

- explicit development mode.

The process creates an Ed25519 key pair and derives a signing-key fingerprint from the public SPKI representation. Verification checks expected key ID, signature, expiry and optional expected protected-content digest.

## Fail-closed behavior

No seal is issued when:

- upstream receipt is unregistered;
- content digest, decision, policy or destination mismatches the receipt;
- upstream decision is not allow;
- receipt is missing;
- destination is blocked;
- ContextFence returns review or block;
- supported policy/action mapping is absent.

## Production replacement

The current key and registry are process-local and unsuitable for multi-instance authority, durability, rotation or recovery. Production requires:

- control-plane-authoritative receipt verification;
- KMS/HSM-backed signing key;
- pinned verification key/trust bundle;
- tenant, policy and key version in payload;
- durable replay/revocation state;
- workload identity and separate signing permission;
- key rotation and historical verification;
- independently retained issuance evidence;
- multi-region consistency and disaster recovery;
- protocol/version governance and external security review.

# 14. SafeAction broker demonstration

SafeAction accepts only an EgressSeal and protected text. It verifies signature, expiry and digest, then derives the only allowed action from the sealed policy.

Current synthetic actions are:

| Policy           | Action                                      | Connector                   |
|------------------|---------------------------------------------|-----------------------------|
| Healthcare       | Reserve synthetic virtual-care slot         | Demo scheduling connector   |
| Finance          | Open synthetic dispute review               | Demo banking case connector |
| Insurance        | Create synthetic claim-review task          | Demo claims connector       |
| Contact Center   | Create synthetic refund review              | Demo CRM connector          |
| Internal Copilot | Create synthetic restricted incident ticket | Demo ticketing connector    |

The broker extracts protected token references, performs no real reidentification and returns a signed synthetic action receipt stating:

- parent seal;

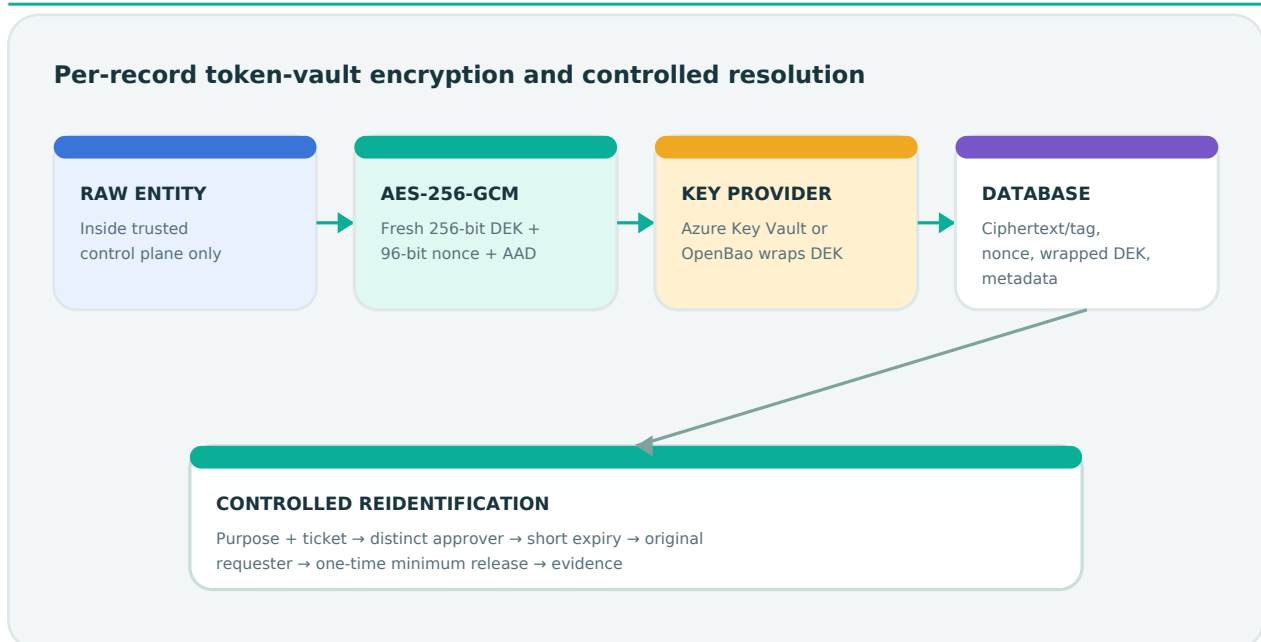
- action and connector;
- destination;
- protected token references;
- raw values visible to model = false;
- connector-only synthetic resolution;
- simulated success and execution time.

Wrong action, invalid seal, changed content, expired seal or blocked public destination returns denial.

## Production broker requirements

- separate infrastructure identity and network boundary;
- field- and operation-level connector policy;
- no generic token-resolution method;
- minimum-value direct release to system of record;
- idempotency/concurrency control;
- user and business authorization;
- approval/step-up/escalation rules;
- connector-specific schemas and validation;
- output/response DLP;
- immutable action evidence;
- transaction reconciliation and rollback;
- incident kill switch and destination revocation.

# 15. Token-vault cryptography



## Implemented mapping encryption

- 1 Compute contextual keyed HMAC lookup over tenant, session, entity type and raw value.
- 2 Reuse an unexpired mapping within that tenant/session context when found.
- 3 Generate an unrelated token with a random hexadecimal suffix.

- 4 Generate a fresh 32-byte DEK.
- 5 Generate a fresh 12-byte nonce.
- 6 Build AAD from `airshield:v1`, tenant, session, token and entity type.
- 7 Encrypt the raw value with AES-256-GCM.
- 8 Ask the selected provider to wrap the DEK.
- 9 Store ciphertext (with the GCM tag appended by the library), nonce, wrapped DEK, wrap key ID, lookup HMAC and expiry.
- 10 Use tenant-scoped constraints and predicates.

AES-GCM provides authenticated confidentiality for stored mapping values and fails if ciphertext/tag/nonce/AAD is altered. AAD helps prevent cross-context ciphertext substitution.

## Key providers

- Azure Key Vault for managed identity and customer-controlled cloud key custody.
- OpenBao Transit for portable Kubernetes separation.
- Local key provider for tests/development only; production startup rejects it.

## Current limitations

- no explicit `crypto_version` or `aad_version` columns;
- token suffix currently has 32 random bits and should be expanded for production margin;
- HMAC index key is application-configured rather than a complete per-tenant KMS hierarchy;
- KMS wrap/unwrap infrastructure identity separation is incomplete;
- no hardware-AES startup check;
- Python cannot guarantee zeroization of immutable plaintext/key buffers;
- no hybrid post-quantum wrapping/signing;
- mass rewrap, rotation and backup/restore coverage requires expansion.

Encryption does not protect detector misses, pre-encryption browser content, compromised authorized key identities, raw logs, bypass paths or data already sent externally.

# 16. Evidence, deletion and reidentification

## Evidence

Evidence records contain tenant sequence, event type, metadata payload, previous hash, event hash, signature, key ID/algorithm and time. A tenant ledger state tracks sequence and tail. The exporter verifies continuity/signatures before writing a private JSONL package and SHA-256 sidecar.

The sidecar is not an immutable anchor. Production requires transfer to independently administered retention-locked storage with external timestamp/object version.

## Deletion

The control plane exposes tenant/session data deletion and retention jobs. Production schedules must separately cover raw transient buffers, encrypted mappings, sessions, receipts, backups, model caches/logs and exported evidence.

## Reidentification

Implemented API flow:

- 1 requester submits mapping, purpose and ticket;
- 2 a different approver approves before short expiry;

- 3 only the original requester retrieves once;
- 4 mapping is decrypted inside the authorized control-plane path;
- 5 metadata evidence records the privileged operation.

API-level dual control does not alone prevent collusion. Production requires separate roles/identities, approval governance, anomaly monitoring and connector-level minimum release.

## 17. Authentication, authorization and tenancy

The existing `control-plane/app/auth.py` approach is retained. The intended production path validates OIDC issuer, audience, signature, expiry, subject, tenant and required roles/scopes. Portable Kubernetes uses exact subject-to-tenant/scope bindings rather than wildcard subjects. Azure uses Workload Identity and token exchange for the configured control-plane scope.

Development auth is explicit and rejected in production. Server-side Next routes do not forward arbitrary browser bearer headers as workload identity. Tenant predicates and composite relationships are used across sessions, mappings, identity bindings, evidence, idempotency and reidentification data.

Production requires identity-provider onboarding, revocation behavior, group/role governance, workload-token rotation, privileged-access management and tenant-isolation penetration testing.

## 18. API and integration surface

### Next same-origin routes

| Route                         | Purpose                                                       |
|-------------------------------|---------------------------------------------------------------|
| <code>/api/health</code>      | UI/control-plane readiness and explicit development mode      |
| <code>/api/sessions</code>    | Protection session creation                                   |
| <code>/api/protect</code>     | Server-side protection proxy or labelled development detector |
| <code>/api/summarize</code>   | Protected-content-only local AI summary path                  |
| <code>/api/egress-seal</code> | Development issue, verify and execute protocol                |

### Control-plane routes

- liveness/readiness;
- create session;
- protect text;
- bind authenticated speaker track;
- delete session data;
- request/approve/retrieve controlled reidentification;
- retrieve and verify evidence.

### Voice routes

- gateway `/ws/voice` for authenticated external browser/non-browser streams;
- edge `/ws/voice` for private processing;
- edge health, protect and bounded file-transcription routes.

## Application integration

The repository includes OpenAPI, AsyncAPI and examples for Java, .NET, Python, Node.js and Go. Supported patterns include SDK call, reverse proxy, sidecar, event consumer/producer and approved media connector. Production SDKs require release versioning, retries/idempotency, telemetry policy and supported-language lifecycle.

# 19. Deployment architecture

## Local hackathon

Docker Compose runs web, control plane, migration job, PostgreSQL and voice edge, with optional Ollama. It intentionally uses visible development credentials, local keys and insecure localhost WebSocket support. It must never be promoted as production configuration.

## Azure-private reference

- AKS
- Azure Workload Identity
- separated Key Vault responsibilities
- PostgreSQL Flexible Server
- private endpoints/DNS/network policy direction
- certificate/CA and preloaded model requirements
- Bicep reference assets

## Cloud-neutral Kubernetes

- Kubernetes workload identity
- OpenBao Transit
- private PostgreSQL with verified TLS
- certificate automation
- NetworkPolicy-capable CNI
- persistent read-only model storage
- independent evidence destination
- Kustomize overlays

All manifests contain customer/release placeholders and require render, policy, image-signature, quota, region, DNS, certificate, firewall, backup and disaster-recovery validation.

# 20. Security-control summary

| Control objective                         | Implemented mechanism                             | Required production evidence                  |
|-------------------------------------------|---------------------------------------------------|-----------------------------------------------|
| Prevent direct external voice edge access | Authenticated gateway and private edge credential | Network tests and direct-edge denial          |
| Prevent client identity forgery           | Signed host claims; discard client trust headers  | Issuer/channel binding tests                  |
| Prevent unauthenticated protect calls     | Workload OIDC/scoped development mode             | Identity/role/tenant negative tests           |
| Prevent cross-tenant mapping access       | Tenant AAD, predicates and constraints            | Isolation and migration tests                 |
| Protect stored mappings                   | AES-256-GCM and wrapped DEKs                      | KMS roles, rotation/recovery and tamper tests |

| Control objective                     | Implemented mechanism                          | Required production evidence                        |
|---------------------------------------|------------------------------------------------|-----------------------------------------------------|
| Prevent provisional egress            | <code>safe_for_egress=false</code> until final | End-to-end message-policy tests                     |
| Bind destination/content release      | EgressSeal protocol                            | Production trust anchor and replay/revocation tests |
| Detect mosaic risk                    | ContextFence factors/threshold                 | Calibrated evaluation and governance                |
| Prevent model-side generic resolution | SafeAction allowlist/broker design             | Real connector penetration and authorization tests  |
| Detect evidence tampering             | Signed sequence/hash chain                     | External immutable anchor and periodic verification |
| Fail closed on missing dependencies   | Production configuration/runtime guards        | Dependency-outage game days                         |

## 21. Threat model and residual risk

| Threat                                    | Current/target control                                             | Residual risk                                           |
|-------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------|
| Application bypasses AirShield            | Gateway/proxy integration and target network egress policy         | Cannot be solved by library code alone                  |
| Detector/ASR misses identifier            | Layered models, final recheck and evaluation target                | No perfect recall guarantee                             |
| Safe facts reconstruct identity           | ContextFence and lower-risk destination                            | Current weights are not production-calibrated           |
| Client forges EgressSeal inputs           | Process-local receipt registry matching content/policy/destination | Production authority still required                     |
| Seal replay to different destination      | Destination ID/route bound into signature                          | Revocation and distributed replay store not implemented |
| Client changes protected content          | SHA-256 digest and signature verification                          | Compromised signer remains high risk                    |
| Model requests arbitrary token resolution | One sealed action and target narrow broker                         | Real connector not implemented                          |
| KMS identity compromise                   | External key custody and scoped roles                              | Authorized compromise can decrypt/sign                  |
| Insider reidentification                  | Requester/approver/purpose/expiry/one-time result                  | Collusion and infrastructure separation remain          |
| Prompt injection/tool abuse               | Target action allowlist and output policy                          | Complete output/tool DLP not implemented                |
| Model supply-chain compromise             | Pinned artifact manifest/hash                                      | Build/release infrastructure also must be trusted       |
| Evidence deletion/reordering              | Signed hash chain and external anchor target                       | Local chain without independent anchor is insufficient  |
| Denial of service                         | Size/rate/session/inference bounds                                 | DDoS/capacity requires environment testing              |
| Compromised user endpoint                 | Host controls and minimized retention                              | AirShield cannot secure a fully compromised endpoint    |



## 22. Compliance and regulatory posture

---

AirShield can support control objectives relevant to data minimization, access control, encryption, evidence, retention and controlled disclosure. Applicability depends on the customer's legal role, country, sector, data flow, model use, contracts and operations.

Potentially relevant frameworks may include GDPR, national NIS2 implementations, DORA, Cyber Resilience Act, EU AI Act, EHDS, HIPAA/BAA obligations, PCI DSS responsibilities, SOC 2 and ISO standards. The repository does not establish conformity or certification. HHS does not recognize a private HIPAA Security Rule certification, and PCI validation method/scope are determined by enforcing entities and assessors—not application source code.

## 23. Validation evidence for this baseline

---

Completed for baseline [d9d83cd](#):

- `npm ci` completed from lock file;
- npm high-severity audit reported zero vulnerabilities;
- TypeScript typecheck passed;
- Next.js production build passed, including `/api/egress-seal`;
- 32 Python control-plane tests passed;
- Ruff format/lint and mypy passed for control-plane scope;
- development API smoke verified protect → registered receipt → EgressSeal issue → verify → SafeAction;
- forged/unregistered receipt was denied;
- changed protected content was denied;
- destination mismatch/replay was denied;
- wrong SafeAction was denied;
- public destination was blocked;
- PDFs were parsed and page/text validated;
- generated/cache paths and common secret signatures were excluded/scanned before commit.

Not executed in this environment:

- real production model inference qualification;
- Docker image builds in a production registry;
- live Azure subscription or Kubernetes cluster deployment;
- external OIDC provider integration;
- real KMS/HSM/OpenBao operations against production infrastructure;
- real microphone/model quality across representative accents/codecs;
- load, DDoS, failover, restore, rotation and penetration tests;
- real SafeAction connector transaction.

## 24. Production limitations and required decisions

Technical authorities must not approve real data until the following are resolved:

- representative legally sourced ASR/detector/diarization/end-to-end evaluations;
- comprehensive output/TTS/tool-parameter protection;
- external EgressSeal trust anchor and authoritative receipt verification;
- durable replay/revocation and key/version protocol;
- production-calibrated ContextFence governance;
- independently secured SafeAction broker and first real connector;
- explicit crypto/AAD schema versions and stronger token identifiers;
- per-tenant key/index hierarchy and infrastructure identity separation;
- model artifact promotion/signing/attestation process;
- PostgreSQL concurrency, tenant isolation, backup/restore and migration validation;
- certificate, secret, signing-key, wrap-key and model rotation drills;
- immutable evidence anchoring and verification operation;
- retention, deletion, subject-rights and backup policies;
- network egress enforcement preventing bypass;
- security/privacy/legal/model/penetration and applicable sector review;
- SLO, capacity, failover, support and incident ownership.

## 25. Recommended authority gates

| Gate         | Approval question                                                             | Minimum evidence                                 |
|--------------|-------------------------------------------------------------------------------|--------------------------------------------------|
| Architecture | Are all raw/protected/action paths and trust boundaries explicit?             | Reviewed data-flow and threat model              |
| Identity     | Can tenant/workload/participant/channel claims be trusted and revoked?        | OIDC and channel-binding tests                   |
| Model        | Are ASR/detection/diarization quality and leakage acceptable by slice?        | Representative evaluation report                 |
| Cryptography | Are key custody, versioning, rotation, recovery and separation sufficient?    | KMS/HSM design and drills                        |
| Data         | Are retention, deletion, backups and evidence legally/operationally governed? | Approved schedules and restore/delete tests      |
| Egress       | Can applications bypass the gateway or destination registry?                  | Network and negative-route tests                 |
| Action       | Can the model expand scope or resolve arbitrary identity?                     | Broker/connector penetration and allowlist tests |
| Operations   | Can the service fail closed while meeting availability needs?                 | SLO, game days, DR and on-call plan              |
| Assurance    | Are claims accurate and independently reviewed?                               | Security/privacy/legal/model sign-offs           |

## 26. Recommended next implementation sequence

---

- 1 Move EgressSeal issue/verification into the Python control plane with externally configured signing/verification keys and versioned schema.
- 2 Build a destination registry with identity, contract, region, data-class and action policy.
- 3 Create a real isolated SafeAction broker for one low-risk reversible connector.
- 4 Add output/TTS and tool-argument DLP.
- 5 Expand vault schema/versioning, token entropy and per-tenant key hierarchy.
- 6 Build representative ContextFence and entity-leakage evaluation corpora.
- 7 Add immutable evidence anchoring and independent verification service.
- 8 Enforce network-level approved AI egress.
- 9 Run one controlled sector pilot with no autonomous high-risk decision.
- 10 Complete independent review and operational readiness before production.

## 27. Authority conclusion

---

The baseline demonstrates a coherent end-to-end product and working development protocols: voice/text protection, destination-specific risk, signed EgressSeal verification and token-only SafeAction. The implementation is materially more than a visual mockup, but its new signing, risk and action layers are deliberately marked as development demonstrations.

A technical authority should approve continued controlled-pilot engineering—not real regulated production—subject to the gates in this dossier.

**Release principle: useful meaning may leave only after authenticated protection, destination authorization, cumulative-risk evaluation and verifiable evidence. Identity resolution and operational action occur only in a separately authorized connector path that does not expose raw values to the model.**